

Sai

UCSH-201

AVL TREE

- Adelson-Velskii & Landis
- BST + Balance criteria
- $|H_L - H_R| \leq 1$
- 4 Cases of imbalance
 - L S L H (Right Rotation)
 - L S R H (Left Rot. on Left Child + Right Rotation)
 - R S L H (Right Rot. on Right Child + Left Rotation)
 - R S R H (Left Rotation)
- Implementation !

Sai

struct Node

```
{  int val ;  
    struct Node *l, *r ;  
    int h ;  
};
```

int height (struct Node *root)

```
{  
    if (root == NULL)  
        return (-1);  
    return (root → h);  
}
```

int max (int x, int y)

```
{  
    if (x > y)  
        return (x);  
    return (y);  
}
```

Sai

```
struct Node * createNode (int v)
```

```
{  
    struct Node *n = NULL;  
    n = (struct Node *) malloc(  
        sizeof(struct Node) );
```

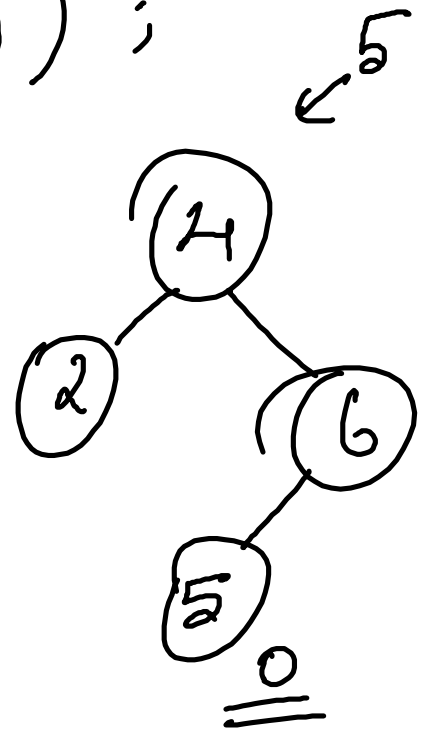
```
    n → val = v;
```

```
    n → l = NULL;
```

```
    n → r = NULL;
```

```
    n → h = 0;
```

```
    return (n);
```



```
↓  
struct Node * Insert (struct Node * root,  
                        int v)
```

```
{  
    if (root == NULL)  
        return (createNode(v));
```

```
if ( v < root → val )  
{  
    Insert(2
```

else if ($v > \text{root} \rightarrow \text{val}$)

{
 $\text{root} \rightarrow \text{r} = \text{Insert}(\text{root} \rightarrow \text{r}, v);$
 if ($(\text{height}(\text{root} \rightarrow \text{r}) - \text{height}(\text{root} \rightarrow \text{l}))$
 $== 2$)

{
 if ($v > \text{root} \rightarrow \text{r} \rightarrow \text{val}$)
 // R S R H
 return (Left Rotate (root));

 else if ($v < \text{root} \rightarrow \text{r} \rightarrow \text{val}$)
 // R S L H
 return (R L Rotate (root));

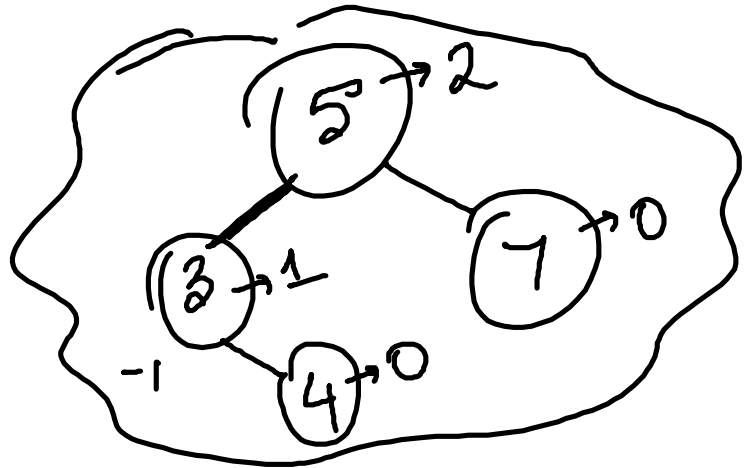
}

}

$\text{root} \rightarrow h = \max(\text{height}(\text{root} \rightarrow l), \text{height}(\text{root} \rightarrow r)) + 1;$

return (root);

}
 max(0, 0) root
 int main() ~~return~~ 5000



{ struct Node *root = NULL;

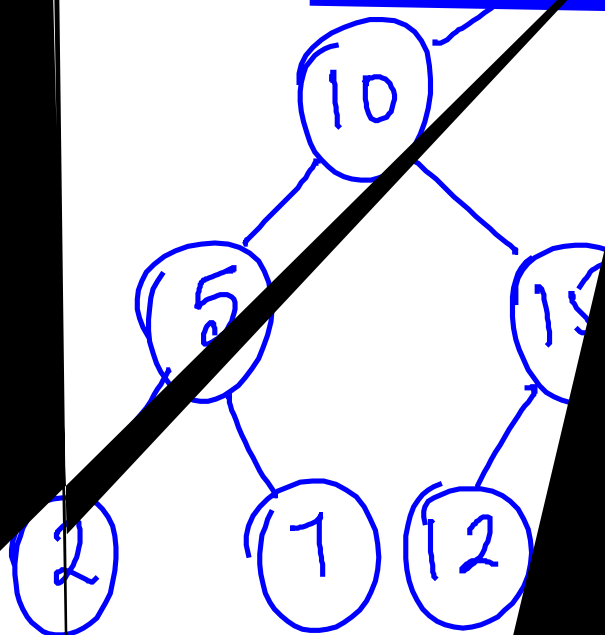
root = Insert(root, 5);

 " " " (" , 7);

 " " " (" , 3);

 " " " (" , 4);

}



Node is Root Node

Skipped Node (root)



→

max

(root)

$h = \max(h_{left}, h_{right}) + 1$

p

Qai

St

Exai

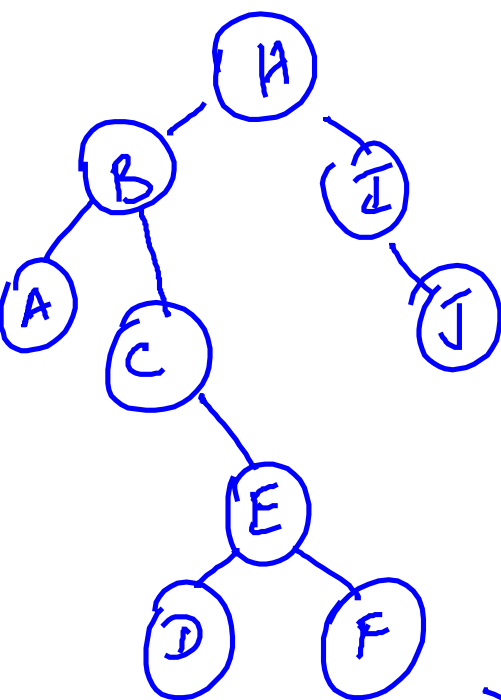
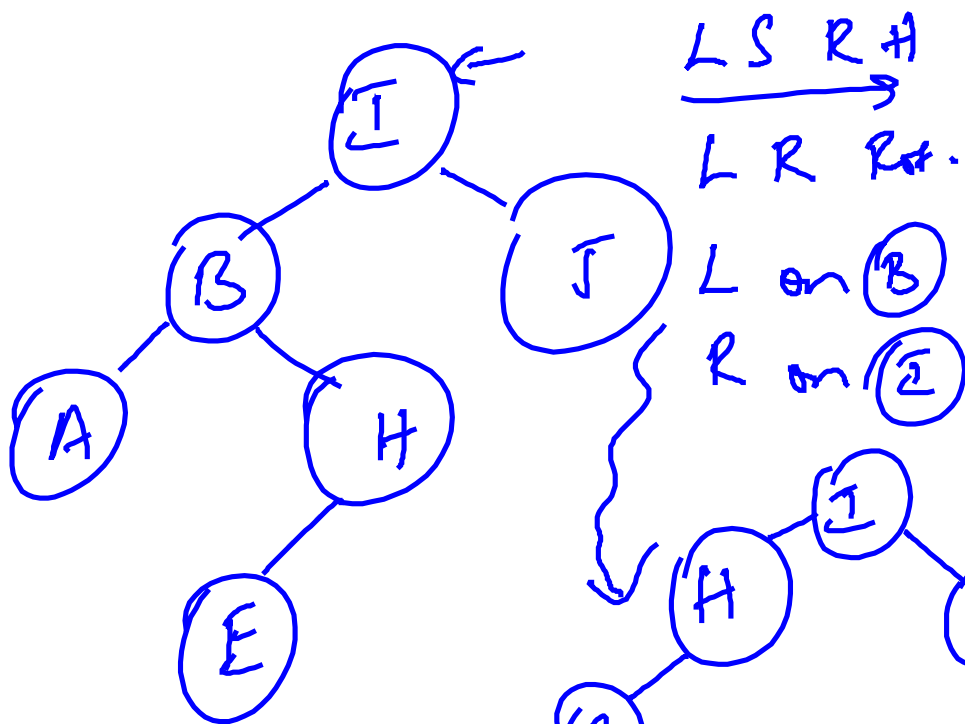
Struktur Node * LR Rotate (Struktur Node * root)

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

1. Perform Left Rotate on the child, followed by Right Rotate the node of imbalance.

2. root = Left Rotate (root → l);

3. return Right Rotate (root);



$$\begin{array}{c} R S L H \\ \hline R L Rot. \end{array}$$

R. on (E)
 L on (B)

